Static Linking

- Static linking means **all required library code is copied into the final executable file** at compile time.
- The executable becomes **self-contained**.

● How it Works

- Object file (`.o`) is created.
- Required library functions (like `printf`) are copied from **static library (.a)**.
- Everything is combined into one executable.

● Characteristics

- Larger executable size
- No need for external libraries at runtime
- Faster execution (no runtime linking)
- More memory usage
- If library updates, program must be recompiled

Dynamic Linking

- Dynamic linking means **libraries are not copied into the executable**.
  Instead, they are linked **at runtime**.
- The program uses **shared libraries (.so)**.

● How it Works

- Executable contains only reference to library.
- At runtime, OS loads required shared library.
- Linking happens while program runs.

● Characteristics

- Smaller executable size
- Saves memory (shared among programs)

- Library updates automatically apply
- Slightly slower at startup
- Needs required library installed on system

Static Linking vs Dynamic Linking

No.	Static Linking	Dynamic Linking
1	Libraries are copied into executable at compile time	Libraries are loaded at runtime
2	Uses static libraries (.a)	Uses shared libraries (.so)
3	Executable size is large	Executable size is small
4	No external dependency during execution	Requires shared libraries on system
5	Faster startup time	Slightly slower startup
6	More RAM usage (each program has its own copy)	Less RAM usage (shared among programs)
7	Library update requires recompilation	Library update does not require recompilation
8	More portable (runs even if library not installed)	Less portable (needs correct library version)
9	Better for embedded systems	Better for large systems & multi-user systems
10	Harder to patch security issues	Easier to patch (update library once)
11	Debugging is simpler (everything inside one file)	Debugging can be complex
12	No version mismatch problem	Version mismatch may cause runtime error
13	Suitable for small standalone applications	Suitable for large enterprise applications
14	Build time slightly longer	Build time slightly shorter

```
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/static$ ls
add.c  divi.c  header.h  main.c  multi.c  sub.c
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/static$ cc -c add.c divi.c multi.c sub.c
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/static$ ls
add.c  add.o  divi.c  divi.o  header.h  main.c  multi.c  multi.o  sub.c  sub.o
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/static$ ar -rcvs libstatic.a add.o divi.o multi.o sub.o
a - add.o
a - divi.o
a - multi.o
a - sub.o
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/static$ ls
add.c  add.o  divi.c  divi.o  header.h  libstatic.a  main.c  multi.c  multi.o  sub.c  sub.o
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/static$ cc -static main.c libstatic.a
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/static$ ls
add.c  add.o  a.out  divi.c  divi.o  header.h  libstatic.a  main.c  multi.c  multi.o  sub.c  sub.o
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/static$ ./a.out
Enter the two number..
5 6
Enter the opeion:-
1-> addition
2->subtraction
3->multiplication
4->addition
 1
addition is:- 11
Enter the opeion:-
1-> addition
2->subtraction
3->multiplication
4->addition
```

Static Linking

```
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/dynamic$ ls
add.c  divi.c  header.h  main.c  multi.c  sub.c
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/dynamic$ ls
add.c  divi.c  header.h  main.c  multi.c  sub.c
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/dynamic$ cc -c sub.c divi.c multi.c add.c
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/dynamic$ ls
add.c  add.o  divi.c  divi.o  header.h  main.c  multi.c  multi.o  sub.c  sub.o
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/dynamic$ cc -shared -o libdynamic.so add.o sub.o multi.o divi.o
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/dynamic$ ls
add.c  add.o  divi.c  divi.o  header.h  libdynamic.so  main.c  multi.c  multi.o  sub.c  sub.o
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/dynamic$ cc main.c ./libdynamic.so
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/dynamic$ ls
add.c  add.o  a.out  divi.c  divi.o  header.h  libdynamic.so  main.c  multi.c  multi.o  sub.c  sub.o
jambaliya@vvdn-Latitude-3420:~/practice/training/task_2/dynamic$ ./a.out
Enter the two number..
3 7
Enter the opeion:-
1-> addition
2->subtraction
3->multiplication
4->addition
 1
addition is:- 10
Enter the opeion:-
1-> addition
2->subtraction
3->multiplication
4->addition
 2
addition is:- -4
Enter the opeion:-
1-> addition
2->subtraction
3->multiplication
4->addition
```

Dynamic Linking